

Repo Orientation Cheat Sheet

Use this at the start of a Codespace or GitHub repo interview. Goal: get oriented fast, narrate clearly, and find the likely breakpoints before touching fixes.

Mental model Shape → Start → Supply → Ship → Signals		5 questions What is it? How does it start? What does it need? How is it deployed? Where will it fail first?
Step	What you are extracting	What to say
1. Shape	Map the repo. Find app code, Docker/build files, K8s/Helm manifests, CI, README, and whether it is one service or several.	"I'm getting the repo shape first: app code, build/container config, and deployment config."
2. Start	Find the real startup path: entrypoint, main file, Dockerfile, command/args, listening port, health endpoints, and startup dependencies.	"Now I'm locating how the app actually starts: process, port, health endpoints, and startup dependencies."
3. Supply	Find inputs and dependencies: env vars, ConfigMaps, Secrets, URLs, DB connection info, mounted files, tokens, and anything mandatory.	"Next I'm checking the input surface: env vars, config, secrets, and upstream dependencies."
4. Ship	Trace how Kubernetes runs and exposes it: Deployment, labels, Service selector, port/targetPort, probes, Ingress host/path/backend.	"Now I'm tracing how Kubernetes is expected to run and expose the app."
5. Signals	Preload the evidence: what logs matter, what health endpoints exist, what pod events would show, and how to prove pod vs service vs ingress quickly.	"Finally, I want the quickest signals so I know where the truth will show up first."

Fast command flow pwd ls tree -L 2 find . -maxdepth 2 -type f sort cat README.md cat Dockerfile find k8s -maxdepth 2 -type f sort Then inspect App entry file • Deployment • Service • Ingress • ConfigMap / Secret What not to do Do not read the whole codebase. Do not explain business logic deeply. Do not open every file. Do not start fixing before you know startup path and K8s wiring.	By the end, you should know • Language/runtime and main file • How the container starts • Port and health endpoints • Required env vars, secrets, and dependencies • Deployment / probes / Service / Ingress wiring • The fastest likely failure points Ultra-short narration script "I'm mapping the repo first." "Now I'm checking how the app starts." "Next I'm checking config, secrets, and dependencies." "Now I'm looking at Deployment, probes, Service, and Ingress." "At this point I should know the quickest likely failure points." Memorize this What is it? How does it start? What does it need? How is it deployed? Where will it fail first?
---	---